The Ch(e)at Code

Agent Patterns for Copilot Chat

▲ ▲ ▼ ▼ ◀ ▶ ◀ ▶ B A START

ISSUE #001 • MARCH 23, 2026 • WEEKLY

Welcome to **The Ch(e)at Code** — a weekly pattern library for landing agents in Copilot Chat. Each issue gives you one proven pattern, the scenario it solves, and the fastest path to landing it with customers.

This issue's pattern: How to handle large PDF files in agent scenarios. Enterprise customers have massive PDFs — regulatory filings, compliance reports, supply chain documentation — that exceed what standard agent configurations can process. Here's the pattern that solves it and how you can replicate it today.

🎮 AGENT SPOTLIGHT

Large PDF Document Agent

Built by Cristiano Almeida Gonçalves • Validated with Coca-Cola

The scenario: Your customer stores hundreds of large PDFs in SharePoint — regulatory filings, compliance documentation, vendor contracts. They need to query across these documents, extract insights, and get answers fast. Standard M365 Copilot handles short documents well, but large PDFs require a different approach.

The solution: An Azure-hosted agent that ingests large PDFs from SharePoint, processes them through Azure AI, and surfaces results inside M365 Copilot and Teams. The agent handles document sizes that would otherwise hit token or file-size limits by offloading processing to Azure infrastructure.

What makes this pattern reusable:

- **Any large-document scenario** — regulatory, legal, financial, supply chain

- **Rebrand in minutes** — swap the system prompt and redeploy for a new customer
- **Full Azure lifecycle** — `azd up` to deploy, `azd down` to tear down and control costs
- **Ships inside Teams** — users interact with the agent in their existing workflow

> ⚠ **Know before you demo:** PDF upload requires an authenticated M365 Copilot or Teams context. It won't work from an unauthenticated browser session. PDFs work reliably today; Office document parity is still catching up.

🧩 **PATTERN BREAKDOWN**

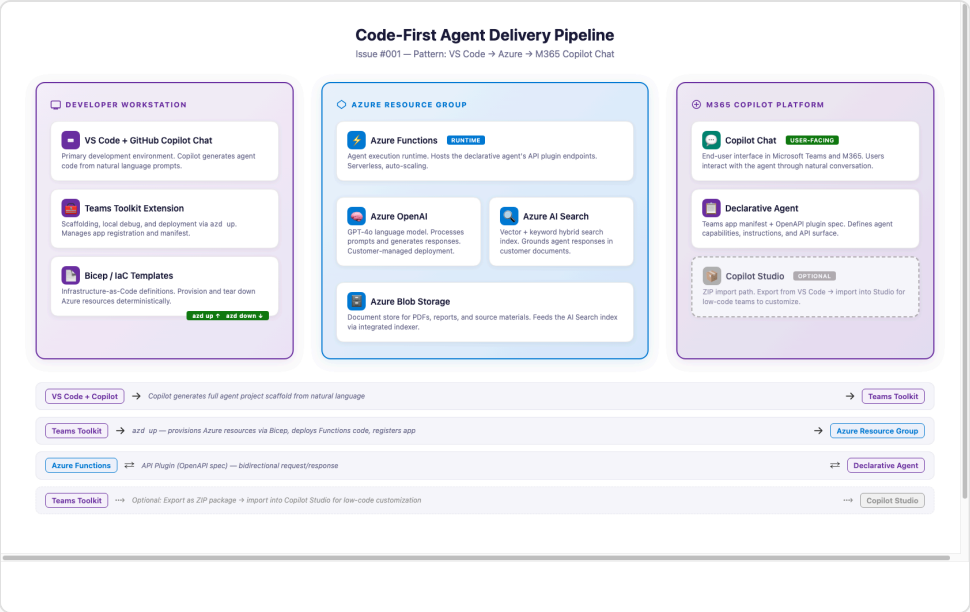
Code-First Agent Delivery

VS Code → Azure → M365 Copilot & Teams

Why this pattern exists: Large PDF scenarios need Azure-hosted processing that goes beyond what Copilot Studio's low-code authoring can deliver. The code-first approach gets you from customer problem to deployed agent faster because you describe what you need in natural language and let Copilot generate the infrastructure.

The delivery pipeline:

```
Describe the scenario in VS Code
↓
Copilot generates agent + Bicep IaC
↓
azd up provisions Azure + deploys
↓
Publish Teams manifest → live in M365
```



Code-First Agent Delivery — from VS Code prompt to live M365 agent

Why CSAs should adopt this pattern:

- **Build in minutes, not days.** One prompt generates the agent, infrastructure, and manifest.
- **Reuse across customers.** Change the system prompt and redeploy — same agent, new brand.
- **Control costs.** `azd down` tears everything down after demos or testing.
- **Skip the low-code ceiling.** Production PDF agents need Azure-level processing. This gets you there directly.

> **Alternate path:** If the customer wants Copilot Studio specifically, you can still use this approach. VS Code generates a ZIP package that imports directly into Copilot Studio. Schema mismatches are flagged clearly and can often be auto-fixed by Copilot.

🛠️ **TRY THIS NOW**

Land This Pattern with a Customer This Week

1. **Identify the scenario.** Which customer has a large-PDF problem? Regulatory filings, compliance docs, vendor contracts, technical manuals — any heavy document workflow in SharePoint qualifies.

2. **Generate the agent.** Open VS Code with Copilot in Agent mode. Prompt: *"Create an agent that analyzes large PDF documents from SharePoint. Deploy to Azure with Bicep and integrate with Teams."* Copilot scaffolds the agent, infrastructure, and manifest.

3. **Customize for the customer.** Update the system prompt with their industry context and document types. This is what makes it feel purpose-built rather than generic.

4. **Deploy and demo.** Run `azd up` to provision everything in Azure. Publish the Teams manifest via dev.teams.microsoft.com. Demo within an authenticated Teams or M365 Copilot session.

5. **Clean up.** Run `azd down` after the demo. No lingering Azure costs. Next customer? Update the prompt and `azd up` again.

💡 **Efficiency tip:** Once you've built this for one customer, the next one takes minutes. The agent is a configurable asset — the only thing that changes between customers is the system prompt. This team has already validated the pattern across multiple accounts.

🏆 FIELD STORIES: WHERE THIS PATTERN LANDS

Where This Pattern Lands

This pattern was validated against a Coca-Cola scenario: large regulatory and compliance PDFs scattered across SharePoint sites that required manual review. The agent was built, deployed to Azure, and integrated into Teams in a single working session.

Customer scenarios where this pattern applies:

- **Regulated industries** — financial services, healthcare, energy with compliance doc review
- **Legal teams** — contract analysis, policy review, due diligence across large document sets
- **Supply chain operations** — vendor documentation, spec sheets, audit reports
- **Technical organizations** — engineering manuals, safety documentation, RFP responses

How to position it: Lead with the customer's document problem, not the technology. Then show speed — "we can have a working agent analyzing your documents inside Teams by the end of this session." Close with cost efficiency — Azure resources spin up only when needed.

Got an agent or pattern to share?

Reply to this email and you might be featured in Issue #002.

The Ch(e)at Code

AIWF Team • AI Business Solutions

Questions or ideas for the next issue? Reply to this email.

Microsoft Internal • Not for external distribution